Experiment 1

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main()

{

   char command[100];


   printf("Enter a Linux command: ");

   scanf("%s", command);


   pid_t pid = fork();
```

```
if (pid < 0)
{
   perror("fork failed");
   return 1;
}
else if (pid == 0)
{
   printf("Child Process\n");
   printf("Child PID: %d\n", getpid());
   printf("Parent PID: %d\n", getppid());

   execlp(command, command, NULL);

   perror("exec failed");
   exit(1);
}
else
{
   printf("Parent Process\n");
   printf("Parent PID: %d\n", getpid());
   printf("Child PID: %d\n", pid);

   wait(NULL);
```

```
    printf("Child process completed.\n");
  }

  return 0;
}
```

Output: